# Evaluating AutoRecLab: How Prompt Complexity and Role Allocation Shape Autonomous RecSys Experiments

Louis Owie
University of Siegen
Siegen, Germany

Lukas Belke
University of Siegen
Siegen, Germany

## ABSTRACT

AutoRecLab aims to automate the implementation and execution of recommender-systems experiments, but its operational reliability, cost efficiency, and reproducibility remain insufficiently understood. We therefore evaluate how task complexity and model capability affect AutoRecLab v0.0.1 and compare different model-role allocations in terms of cost, runtime, execution status, and output characteristics under a fixed Phase-2 setup. In Phase 1, we benchmarked three model tiers on a simplified exploratory task and a complex end-to-end experiment; in Phase 2, we evaluated four runs covering three model-role allocations across the Planner, Coder, Reviewer, and Summarizer roles. Among the preserved v0.0.1 invocations, technical completion rates for the simplified task ranged from 50% to 100%. For the complex prompt, GPT-5.4 completed one of six invocations and GPT-5.4-mini completed neither of its two preserved invocations, while no GPT-5-nano invocation was preserved. The predominant failure modes involved incorrect LensKit API usage and hallucinated dataset paths. All four Phase-2 runs completed the standardized pipeline, and the two mixed runs cost approximately $2.38 per run, about 76% less than the homogeneous GPT-5.6-terra run, although the latter produced the most detailed analysis. However, the two independently executed mixed runs selected different seeds, hyperparameters, and reporting structures, indicating that technical completion does not ensure scientifically equivalent or reproducible experiments. Due to the limited number of runs and the simultaneous changes between framework versions, the findings characterize operational behavior under the evaluated conditions rather than establish a generally optimal configuration.

## KEYWORDS

Recommender Systems, AI4Research, Reproducibility, Research Automation, Large Language Models, Cost-Efficiency

## 1 INTRODUCTION

### 1.1 Background

Scientific discovery is currently transitioning from manual engineering and research to end-to-end automation, moving toward Artificial Research Intelligence (ARI) [5]. Adjacent fields already utilize multi-agent systems to automate entire research pipelines, occasionally producing papers that successfully pass peer review [5]. Other major systems have contributed significantly to recent research in biomedical fields by formulating hypotheses and novel drug candidates [7]. In contrast, the recommender systems (RecSys) domain still lags behind in automation, though initial research frameworks such as AutoRecLab are emerging to close this gap [5].

### 1.2 Research Problem

A major issue in recommender systems research is that setting up experimental pipelines is still a completely manual task. Although other fields of machine learning use automated frameworks to design experiments and accelerate discoveries [7], a human workflow requires extensive manual engineering. In contrast, an autonomous multi-agent framework reduces the active human input to a few minutes of prompt design, shifting the remaining workload to autonomous execution loops that run between 2 to 9.5 hours on consumer hardware compared to days of work for a manual experiment [5]. Current AutoRecSys tools (e.g., Auto-Surprise or LensKit-Auto) are too narrow and only optimize hyperparameters and algorithm selection, while existing multi-agent research systems like the Sakana AI Scientist [8] already promise this required automation, yet they are too general, lack domain-specific requirements, and struggle with reliability and qualitatively satisfactory results [5]. Expanding these problems, reproducibility is highly difficult because minor pipeline variations, such as data splitting protocols or algorithm configurations, can completely distort evaluation results [5].

To solve this manual bottleneck and combat reproducibility concerns, Beel and Gipp [5] propose a paradigm shift and introduce AutoRecLab, a RecSys-specific multi-agent framework for automated research. In their first case study, the authors reported that three of four end-to-end runs were technically completed. Across the three completed runs, however, only 15 of 27 algorithm–dataset–run combinations were classified as valid. However, while these initial results show that autonomous research automation is possible, a deeper evaluation is needed to understand the system's operational boundaries. Specifically, it remains unclear how framework reliability scales across task complexities, where specific tool-use failure modes occur, and how multi-agent role architectures—such as delegating tasks across heterogeneous model tiers can be leveraged to achieve reliable execution while optimizing computational and monetary costs.

### 1.3 Original Work

Previous approaches to automating recommender systems mainly focus on individual engineering tasks rather than the scientific research process as a whole. AutoRecSys frameworks such as Auto-Surprise, LensKit-Auto, and LibRec-Auto support functions including algorithm selection, hyperparameter optimization, experiment configuration, and repeated offline evaluation [1, 9, 10]. Although these systems reduce the manual effort required to construct recommender system experiments, the underlying research question, experimental design, interpretation, and reporting still remain the responsibility of human researchers.

Recent autonomous research systems demonstrate a broader form of scientific automation. The Sakana AI Scientist, for example, generates research ideas, implements and executes experiments, visualizes results, writes manuscripts, and applies an automated review process [8]. Its successor introduced agentic tree search and produced a fully AI-generated manuscript that passed the review process of a machine-learning workshop [11]. Similarly, Google's AI Co-Scientist uses specialized agents to generate, critique, rank, and refine scientific hypotheses, with initial applications validated in biomedical research [7]. However, independent evaluations have also identified substantial limitations in autonomous experiment execution, novelty assessment, and the reliability of generated results [6].

Motivated by these developments, Beel et al. introduce the concept of an Autonomous Recommender-Systems Research Lab, or AutoRecLab [4]. In contrast to conventional AutoRecSys tools, the proposed vision covers the complete research lifecycle, including problem ideation, literature analysis, experimental design and execution, result interpretation, manuscript drafting, and provenance logging. The authors position AutoRecLab as a domain-specific application of Artificial Research Intelligence that incorporates the methodological requirements of recommender-systems research.

The AutoRecLab v0.0.1 prototype evaluated in this work implements a narrower subset of this broader vision. It accepts a natural-language description of a research task and automatically derives technical requirements for the corresponding experiment. The system then generates several candidate implementations, executes and evaluates them, and uses tree search to iteratively debug or improve promising solutions [5]. Rather than consisting of several independent LLMs that collaborate in parallel, v0.0.1 uses a single configured language model through sequential role-specific prompts for planning, code generation, and review. It does not yet perform autonomous research-idea generation, systematic literature analysis, or manuscript preparation.

The original proof-of-concept study evaluated whether this architecture could autonomously implement complete recommender-systems experiments from natural-language instructions. In its case study, AutoRecLab was tasked with generating executable experimental pipelines, processing recommender-system datasets, training and evaluating algorithms, and interpreting the resulting measurements [5]. The authors reported that three of four end-to-end runs were technically completed. Across these completed runs, 15 of 27 algorithm–dataset–run combinations were classified as valid. The successful executions required only a small amount of initial human input, while the subsequent code generation, execution, debugging, and refinement proceeded autonomously over several hours.

These results provide initial evidence that LLM-driven research automation is technically feasible in the recommender-systems domain, but they do not establish that AutoRecLab is already a complete or consistently reliable autonomous research laboratory. The small number of evaluated runs, the dependence on generated code and external libraries, and the limited scope of the implemented research stages leave the system's operational boundaries unclear. The broader contribution of the original work is therefore twofold: it presents an initial code-centered prototype and proposes a research agenda involving open implementations, standardized benchmarks, transparent research logs, reproducibility standards, and governance mechanisms for future autonomous recommender-systems research [4, 5].

### 1.4 Research Goal

The overarching goal of this work is to systematically evaluate and optimize the operational readiness of the AutoRecLab framework for autonomous recommender systems research. To address the limitations identified in prior work, we structure our investigation into two complementary phases guided by two primary research questions:

- **RQ1 Prompt Complexity & Failure Modes:** How does AutoRecLab v0.0.1 perform across task complexities and model tiers, and what are its primary execution bottlenecks?
- **RQ2 Model-Role Allocation under a Fixed Setup:** How do the evaluated model-role allocations differ in cost, runtime, execution status, and output characteristics under the fixed Phase-2 setup?

This investigation was conducted as part of the Machine Learning Lab [2].

## 2 METHODOLOGY

We evaluate the structural reliability and capabilities of the AutoRecLab framework by conducting a systematic analysis across varying degrees of task complexity, software versions, and model configurations.

### 2.1 Phase 1: AutoRecLab v0.0.1 Baseline Benchmark

To evaluate AutoRecLab v0.0.1 and conclude if its standard configuration needed fine-tuning, we used the following parameters throughout all baseline runs: We set the number of nodes to 3 with 10 iterations, the model temperature to 1, and we fixed the debug probability and epsilon at 0.3. The local automation environment operated on a pinned software stack consisting of `lenskit==0.14.4`, `pandas==2.3.2`, `numpy==1.26.4`, `scipy==1.16.2`, `numba==0.58.1`, and `scikit-learn==1.7.1` as available libraries the agent could use. To test the operational limits of the framework, we exposed it to three local datasets: MovieLens100K, Amazon-VideoGames, and Last.FM, which the model could access in the workspace folder. These datasets cover different recommendation domains and data modalities, ranging from explicit movie and e-commerce ratings to implicit, time-stamped social tagging interactions. We connected the large language models via the OpenAI API and evaluated `gpt-5-nano` as a baseline model, alongside `gpt-5.4-mini` and `gpt-5.4` as the most capable models.

In our initial runs, we tested the prompt (Appendix A.1) also utilized in [5]. This prompt tasked the agent with developing and executing an automated LensKit pipeline to preprocess three implicit feedback datasets, evaluate three recommendation algorithms across five distinct random seeds using a user-based 80/20 holdout split, and statistically analyze the variation in nDCG and precision metrics to quantify the impact of data-splitting randomness. We ran the experiment multiple times with the standard configuration

described above and on the three large language models. Since we struggled to evaluate the system on this prompt, we simplified the prompt.

We reused the example research task described in the original preprint but did not directly replicate its complete four-run protocol. The original experiment used GPT-5, slightly different prompts and variables across runs, and different tree-search configurations. Our Phase-1 experiments used different model variants and a separately fixed configuration. The results therefore represent an evaluation under new experimental conditions rather than a direct replication. A detailed comparison is provided in Table 3 in Appendix B.

The adjusted prompt (Appendix A.2) substantially reduced the execution scope. It tasked the agent to do a basic exploratory analysis on the MovieLens100K dataset with different goals: Identify the top three most active users, identify different items, confirm the number of unique IDs, calculate co-occurrence of two items, and confirm that there is no data outside of the expected time range of the dataset. This prompt does not cover a whole recommender systems research task, but it tests fundamental capabilities the agent should consistently and reliably be able to perform. We ran the simple prompts several times with `gpt-5-nano`, `gpt-5.4-mini`, and `gpt-5.4` under standard configurations.

The original execution archive was incomplete. We therefore report only the preserved AutoRecLab invocations and do not reconstruct missing historical runs. Each invocation is listed separately in the run ledger in Appendix C. For the complex prompt, a complete result required 45 dataset–algorithm–seed combinations (3 datasets, 3 algorithms, and 5 seeds), corresponding to 270 metric values for the two metrics and three cutoffs. The presence of an intermediate result file was not counted as technical completion if the corresponding AutoRecLab invocation terminated with an exception or without an accepted final node.

For the v0.0.1 experiments, monetary costs were tracked manually through the OpenAI Platform billing dashboard, while total runtime and lines of code (LOC) generated per runfile were recorded locally. The codebase for these v0.0.1 baseline runs is available in the original repository (AutoRecLab v0.0.1)[1].

## 2.2 Dev Branch: Model-Role Allocation & Cost Efficiency

To address **RQ2**, we evaluated four runs covering three model-role allocations across the Planner, Coder, Reviewer, and Summarizer roles under the fixed Phase-2 setup (Figure 1). All four Phase-2 runs completed the evaluated workflow. This improvement coincided with an updated software stack, OmniRec integration, different models, additional repair mechanisms, modified tree-search parameters, and a narrower prompt. The present design does not allow us to isolate the contribution of any single change.

To systematically evaluate how multi-agent role delegation impacts execution success and API costs, we introduced a standardized end-to-end pipeline prompt (Appendix A.3). This prompt tasked the agent with preprocessing the MovieLens100K dataset using 5-core filtering, converting ratings greater than 3 into implicit interactions, generating 5 random seed splits for a user-based 80/20 holdout, training ALS and Popularity baselines using standard hyperparameters, measuring nDCG@10 and Precision@10, and performing a short statistical seed-sensitivity analysis.

Across all Dev branch runs, the tree-search parameters were held fixed: 2 draft nodes, a maximum of 8 search iterations, 2 refinement iterations, a request timeout of 120 seconds, and an execution timeout of 5400 seconds. Type checking was enabled with up to 3 repair attempts per node, yielding exactly 13 search nodes per run.

The primary independent variable in Phase 2 was the assignment of LLM backends across the four agent roles: Planner, Coder, Reviewer, and Summarizer. We evaluated four configurations:

- **R1 (Mixed Multi-Agent):** Role allocation using `gpt-5.6-terra` (Planner), `gpt-5.6-luna` (Coder), `gpt-5.4-mini` (Reviewer), and `gpt-5-mini` (Summarizer).
- **R2 (Mixed Multi-Agent Replica):** An identical replicate of R1 to measure intra-configuration variance and non-deterministic behavior.
- **R3 (Solo Terra):** A single-agent setup assigning `gpt-5.6-terra` to all four roles.
- **R4 (Solo Luna):** A single-agent setup assigning `gpt-5.6-luna` to all four roles.

R1 and R2 were independent AutoRecLab invocations. R2 was started in a new run directory without reusing a checkpoint, generated code, or execution state from R1.

Unlike v0.0.1, the Dev version introduces automated cost logging. We tracked execution expenditures directly through the generated cost log files, which record prompt and completion token usage alongside calculated USD costs per agent call. In addition, we recorded execution duration, average tree node scores, and lines of code (LOC) per node. All experiments were executed on consumer workstations running Windows 11. The full codebase, configuration files, and experiment logs for Phase 2 are hosted on our dedicated development branch (feature/per_role)[2]. This branch is a custom version of the Dev version and introduces the LLM-Selection per role feature.

# 3 RESULTS

This section presents the results for the two research questions defined in Section 1.4. First, we address RQ1 by evaluating AutoRecLab v0.0.1 across different prompt complexities and model tiers. We report execution success rates, runtime, API costs, generated lines of code, and the recurring failure modes observed during the baseline experiments. Second, we address RQ2 by evaluating the updated development version of AutoRecLab on a standardized end-to-end recommender-systems task. In this phase, the prompt and tree-search parameters were held constant, while the language models assigned to the Planner, Coder, Reviewer, and Summarizer roles were varied. The resulting configurations are compared in terms of execution success, runtime, monetary cost, node scores, code complexity, and the quality and reproducibility of the generated statistical analyses.

[1] https://github.com/ISG-Siegen/AutoRecLab/releases/tag/v0.0.1

[2] https://github.com/LouisOwie/AutoRecLab_Team7/tree/feature/per_role

### 3.1 AutoRecLab v0.0.1: Prompt Complexity & Failure Modes

We first evaluated AutoRecLab using the original experimental prompt proposed by Beel et al. [5]. The task required the framework to preprocess three datasets, configure and execute three LensKit algorithms, evaluate multiple ranking metrics, and perform a statistical analysis across different random seeds. Despite multiple attempts, the framework struggled to complete the experiment reliably.

Among the preserved complex-prompt invocations, GPT-5.4 was the only model that technically completed the full pipeline and produced all 45 requested dataset–algorithm–seed combinations. One of six GPT-5.4 invocations achieved this status, corresponding to 16.7%. Neither of the two preserved GPT-5.4-mini invocations completed the pipeline. No complex-prompt GPT-5-nano invocation was preserved, so no completion rate can be reported for this model. The completed GPT-5.4 invocation required approximately 60 minutes and incurred API costs of roughly $1.50.

Analysis of the failed runs revealed several recurring failure modes. The most common issue was incorrect usage of the LensKit API. Generated code frequently referenced non-existing functions, invalid constructor parameters, or outdated interfaces. Furthermore, the framework regularly hallucinated file paths instead of using the datasets available in the working directory. These issues often resulted in Python execution errors before the actual experiment could be started. Tool crashes and infinite execution loops occurred only rarely and therefore did not represent the primary bottleneck.

Because the original prompt proved too difficult for reliable evaluation, we introduced a simplified data-analysis task on the MovieLens100K dataset. This prompt required the agent to compute basic dataset statistics and validate several properties of the data. Compared to the original experiment, this task significantly reduced the required domain knowledge and implementation complexity.

**Table 1: Success rates for both prompt types.**

| Model | Complex Prompt | Simple Prompt |
|---|---|---|
| GPT-5-nano | Not assessable | 2/2 (100%) |
| GPT-5.4-mini | 0/2 (0%) | 3/3 (100%) |
| GPT-5.4 | 1/6 (16.7%) | 1/2 (50%) |

Table 1 also reports the technical completion rates for the preserved simplified-task invocations. All three GPT-5.4-mini invocations and both GPT-5-nano invocations technically completed the task, although one GPT-5-nano run used a dataset-path fallback and is therefore marked for caution in the run ledger. One of two GPT-5.4 invocations completed the task; the other process failed to start. The completed outputs were mutually consistent for the requested checks, but no independent reference calculation was performed.

### 3.2 Dev Branch: Role Allocation & Cost Efficiency

To address **RQ2**, we evaluated four runs covering three model-role allocations across the Planner, Coder, Reviewer, and Summarizer roles under the fixed Phase-2 setup (Figure 1). All four Phase-2 runs completed the evaluated workflow. This improvement coincided with an updated software stack, OmniRec integration, different models, additional repair mechanisms, modified search parameters, and a narrower prompt. The present design does not allow us to isolate the contribution of any single change.


**Figure 1: LLM backend assignments across agent roles for Dev branch configurations.**

**Table 2: Overview of execution metrics across Dev branch role configurations.**

| Metric | R1 (Mixed) | R2 (Replica) | R3 (Solo Terra) | R4 (Solo Luna) |
|---|---|---|---|---|
| Completed | Yes | Yes | Yes | Yes |
| Duration (min) | 43.95 | 43.93 | 62.33 | 46.18 |
| Total Cost ($) | 2.38 | 2.39 | 9.84 | 3.64 |
| Avg. Node Score | 0.414 | 0.429 | 0.620 | 0.475 |
| Avg. LOC / Node | 147.5 | 179.8 | 248.5 | 148.3 |

Table 2 summarizes the performance metrics for all four configurations on the Dev branch. The single-agent setup using `gpt-5.6-terra` across all roles (R3) was the most expensive configuration, requiring $9.84 in API costs and taking 62.33 minutes to complete. The single-agent `gpt-5.6-luna` configuration (R4) reduced total costs to $3.64 with an execution time of 46.18 minutes. The mixed multi-agent setups (R1 and R2), which delegated reviewing and summarization tasks to lighter models (`gpt-5.4-mini` and `gpt-5-mini`), achieved the lowest overall cost at approximately $2.38 per run and completed in under 44 minutes.

An analysis of token distribution across individual agent roles in the mixed runs shows that the Coder agent accounted for the majority of API expenditures, representing between 55% and 63% of total costs ($1.40 to $1.57). The Reviewer agent formed the second largest cost component at 33% to 40% ($0.58 to $0.65). In comparison, token consumption by the Planner and Summarizer agents remained low, together contributing less than 5% to total costs.

While all configurations successfully ran the pipeline, output quality and analytical detail varied depending on the model backend. Configuration R3 achieved the highest average tree node score (0.620) and produced the largest scripts (248.5 LOC per node), incorporating detailed statistical metrics such as coefficients of variation and variability comparisons across seeds. Configuration R4

provided 95% confidence intervals, whereas R1 and R2 provided standard descriptive statistics.

Furthermore, we observed behavioral non-determinism between the identical configurations R1 and R2. When tasked with selecting five random seeds, R1 selected non-uniformly spaced values (`[11, 23, 37, 41, 59]`), while R2 selected uniform intervals (`[11, 22, 33, 44, 55]`). Additionally, agents interpreted the instruction for standard hyperparameters differently; for instance, the ALS implicit weight parameter varied from 1.0 in R1 to 40 in R3. As a result, mean ALS nDCG@10 values ranged from 0.160 (R4) to 0.185 (R1), whereas Popularity baseline results remained consistent across runs (0.137 to 0.140). Additionally, agents structured their reports differently: while R1 output absolute metrics per algorithm, R2 autonomously shifted to reporting paired difference metrics (ALS - Pop).

We inspected each Phase-2 run's final generated code.py, execution out.log, per-seed result CSV, and available configuration, metadata/counts, and summary files, and performed a static and numerical audit. The audit checked for the requested dataset, algorithms, metrics, preprocessing operations, and user-wise holdout logic; the generated implementations were not identical across runs, so this was a static inspection rather than a proof of semantic correctness. In each final per-seed result CSV, the five seeds listed in the generated code occurred exactly once for ALS and once for Pop. All expected metric cells were present, no seed-algorithm-metric key was duplicated, and all metric values were finite and within the [0,1] range. We recomputed the across-seed mean, sample standard deviation, minimum, maximum, and range directly from the per-seed CSVs. The values matched the accompanying CSV summaries for R1, R3, and R4; R2's selected artifact had no accompanying summary CSV. Where post-processing counts were recorded, AutoRecLab-generated metadata or logs reported 938 users, 1,008 items, and 54,413 interactions. The result files showed ALS above Pop on both reported nDCG@10 and Precision@10 for every seed. This ordering is a domain-level plausibility check and does not independently verify the preprocessing, splitting, or metric implementations. We did not recompute nDCG@10 or Precision@10 from raw recommendation lists, independently recompute the post-processing counts from the raw dataset, or re-execute the generated code outside the AutoRecLab runtime.

R4 produced a complete result artifact, although its selected node was flagged as buggy by the automated reviewer. We therefore include its reported execution metrics and artifact-completeness results but do not treat the reviewer assessment or node score as evidence of scientific validity.

For complete run-by-run statistical outputs, raw execution logs, and detailed prompt-token breakdowns, we refer the reader to the supplementary artifacts hosted in our repository (feature/per_role).

## 4 DISCUSSION

The two experimental phases provide substantially different assessments of AutoRecLab's operational readiness. Regarding RQ1, AutoRecLab v0.0.1 was strongly affected by task complexity and model capability. The framework performed comparatively reliably on the exploratory data-analysis task but almost entirely failed on the complex end-to-end recommender-systems experiment. Regarding RQ2, all four configurations of the updated development version completed the standardized pipeline, although they differed considerably in cost, runtime, analytical depth, and reproducibility.

The high success rates of the simplified v0.0.1 prompt should not be interpreted as evidence that AutoRecLab v0.0.1 was already reliable for autonomous recommender-systems research. The task primarily required data loading, aggregation, and validation of known dataset properties rather than the construction of a complete experimental pipeline. Within these runs, GPT-5.4-mini showed the most favorable observed trade-off between runtime, cost, and reliability. However, this result is based on only three preserved runs.

The complex prompt exposed a strong dependency on model capability and technical compatibility. GPT-5.4 technically completed one of six runs, corresponding to 16.7%, while the smaller models did not complete the task successfully. The original proof-of-concept technically completed three of four end-to-end runs, corresponding to 75%, but only 15 of 27 algorithm–dataset–run combinations were classified as valid [5]. Our observed completion rate was therefore approximately 58 percentage points lower, but this is not a direct replication comparison: the evaluations differed in model versions, prompts, software environments, search configurations, and success criteria. The original end-to-end completion rate was consequently not reproduced under our experimental conditions.

The dominant failure source in v0.0.1 was incorrect interaction with the installed LensKit version. Generated scripts frequently used outdated interfaces, invalid parameters, or nonexistent functions. Hallucinated dataset paths caused additional failures before the actual evaluation could begin. The primary bottleneck was therefore not necessarily the tree-search architecture itself, but insufficient grounding in the available software environment. Autonomous research systems that depend on external libraries require access to version-specific documentation, structured API information, and validation mechanisms that identify incompatible calls before executing complete experiments.

The development-version results support this interpretation. All four configurations successfully completed the standardized recommender-systems task, in contrast to the v0.0.1 complex-prompt runs. This improvement coincided with the updated LensKit version, OmniRec integration, type checking, and additional repair mechanisms. However, Phase 2 also used different models, tree-search parameters, and a narrower experimental prompt. The improvement can therefore not be attributed to a single framework change. The results only demonstrate that the complete development setup was substantially more reliable than the evaluated v0.0.1 setup.

Under the fixed Phase-2 setup, the two mixed runs achieved the lowest observed costs at approximately $2.38 per run and required about 44 minutes, whereas the homogeneous Terra run cost $9.84 and required more than 62 minutes. This corresponds to an observed cost reduction of approximately 76%. In the evaluated Phase-2 runs, the mixed allocation therefore achieved the lowest observed costs without preventing technical completion.

However, lower cost was associated with less detailed output. The homogeneous Terra configuration achieved the highest average

node score and produced the most comprehensive statistical analysis. The mixed runs generated executable outputs with complete reported result tables, but reported less complete statistical information. Node scores and lines of code should not be interpreted as independent measures of scientific quality, but the results indicate a practical trade-off between analytical depth and monetary cost. The mixed configuration therefore appears cost-efficient rather than universally superior.

The identical mixed configurations R1 and R2 also revealed substantial non-determinism. They selected different random seeds, interpreted "standard hyperparameters" differently, and produced different report structures. The ALS weight parameter also varied across configurations, which contributed to different absolute metric values. Although every Phase 2 run executed successfully, the resulting experiments were not scientifically equivalent. This demonstrates that successful execution alone is insufficient for reproducible autonomous research.

The observed variation extends existing reproducibility concerns in recommender-systems research [3]. In an autonomous system, experimental decisions may be inferred independently during every run. Recording only the initial prompt is therefore not sufficient. Reproducibility requires preserving the generated requirements, selected seeds, hyperparameters, software versions, source code, execution logs, intermediate revisions, and final outputs. The differences between R1 and R2 demonstrate that detailed provenance tracking is a necessary component of autonomous research systems.

A further implication is that autonomy must be balanced with experimental control. Instructions such as "standard hyperparameters" allow the agent to make reasonable decisions, but they also introduce uncontrolled variation. A more reliable approach would combine a natural-language research question with a structured experimental contract defining fixed datasets, preprocessing steps, hyperparameters, seeds, metrics, and required outputs. The agent could remain autonomous in implementation and debugging while being prevented from silently changing variables that affect the scientific interpretation.

Overall, the results partially support the broader AutoRecLab vision [4, 5]. AutoRecLab v0.0.1 demonstrated basic automation capabilities but remained highly unreliable for complex recommender-systems experiments. The updated development setup completed all evaluated pipelines, and the mixed allocation had the lowest observed API costs among the tested Phase-2 runs. Nevertheless, consistent execution did not guarantee equivalent experimental designs or reproducible outputs. AutoRecLab should therefore be considered a promising research prototype that still requires stronger tool grounding, explicit experimental constraints, detailed provenance tracking, and external scientific validation.

## 5 LIMITATIONS

Several limitations must be considered when interpreting our findings. First, the number of runs was small and uneven across models and configurations. In Phase 1, only one successful execution of the complex prompt was obtained, while only two or three preserved simplified-task invocations were available per model. In Phase 2, only the mixed configuration was repeated, whereas the homogeneous Terra and Luna configurations were each evaluated once. The reported success rates and performance differences should therefore be understood as observations from the evaluated runs rather than stable estimates of general model or configuration performance.

Second, the evaluation was restricted to models available through the OpenAI API. For AutoRecLab v0.0.1, this was partly caused by the framework's available backend support rather than by a deliberate exclusion of other providers. Models from Anthropic, Google, or open-source families may behave differently. In addition, API prices and model implementations may change, limiting the long-term comparability of the reported cost measurements.

Third, the AutoRecLab v0.0.1 configuration was kept fixed across models and prompts. Parameters such as the number of nodes, iteration limits, and debugging probabilities were not optimized. This improved comparability between the baseline runs, but alternative configurations may have produced different success rates or costs. The results therefore describe the evaluated standard configuration rather than the maximum achievable performance of AutoRecLab v0.0.1.

Fourth, the simplified Phase 1 prompt represented an exploratory data-analysis task rather than a complete recommender-systems experiment. Its high success rates only demonstrate basic capabilities such as data loading, aggregation, and validation. They should not be interpreted as evidence that AutoRecLab v0.0.1 reliably performs autonomous recommender-systems research.

Fifth, the comparison between AutoRecLab v0.0.1 and the development version was not a controlled ablation study. Phase 2 changed the software stack, LensKit version, models, prompt, tree-search parameters, and experimental scope simultaneously. Consequently, the higher execution reliability of Phase 2 cannot be attributed to a single framework modification. Similarly, the lower costs of the mixed role configuration were observed in only two runs and do not establish that this allocation is generally optimal.

Finally, the Phase 2 experiment covered only MovieLens100K, two algorithms, and two ranking metrics. Some experimental decisions, including seed selection and the interpretation of "standard hyperparameters", were left to the agents and varied across runs. Output reliability was assessed through execution status, artifact inspection, static completeness checks, and aggregate-statistic recomputation, but not through independent validation of the preprocessing, split, or metric implementations against raw data or an external reference. The findings therefore primarily characterize operational reliability, cost, and agent behavior under the evaluated conditions.

## CODE AND DATA AVAILABILITY

The code, prompts, configurations, run artifacts, and final manuscript are available at https://github.com/LouisOwie/AutoRecLab_Team7. The Phase-1 experiments used the official AutoRecLab v0.0.1 release: https://github.com/ISG-Siegen/AutoRecLab/releases/tag/v0.0.1. The Phase-2 experiments used commit `43673ac`.

The Phase-1 and Phase-2 prompts are reproduced in Appendix A and run artifacts are stored in (Phase-1 artifacts) and (Phase-2 artifacts), respectively. These directories contain generated code, execution logs, cost logs, and result files. All reported Phase-2 results are based on the final selected node of each run.

## ACKNOWLEDGEMENTS

AI was used to assist in drafting specific sections of this manuscript based on comprehensive prompts that incorporated relevant research papers. The human authors performed all subsequent editing, fact-checking, and validation, and take full responsibility for every statement in the final text.

## REFERENCES

[1] Rohan Anand and Joeran Beel. 2020. Auto-Surprise: An Automated Recommender-System (AutoRecSys) Library with Tree of Parzens Estimator (TPE) Optimization. In *Proceedings of the 14th ACM Conference on Recommender Systems*. 585–587. doi:10.1145/3383313.3411467

[2] Joeran Beel and Moritz Baumgart. 2026. Intelligent Systems Seminar. *Universität Siegen* (2026).

[3] Joeran Beel, Corinna Breitinger, Stefan Langer, Andreas Lommatzsch, and Bela Gipp. 2016. Towards reproducibility in recommender-systems research. *User Modeling and User-Adapted Interaction* 26 (03 2016), 69–101. doi:10.1007/s11257-016-9174-x

[4] Joeran Beel, Bela Gipp, Tobias Vente, Moritz Baumgart, and Philipp Meister. 2025. From AutoRecSys to AutoRecLab: A Call to Build, Evaluate, and Govern Autonomous Recommender-Systems Research Labs. arXiv:2510.18104 [cs.IR]

[5] Joeran Beel, Bela Gipp, Tobias Vente, Moritz Baumgart, Philipp Meister, and Sinan Pourazari. 2026. AutoRecSys or AutoRecLab? A Call to Build, Evaluate, and Govern Autonomous Recommender-Systems Research Labs, and a First Proof-of-Concept. *ACM Transactions on Recommender Systems* (February 2026).

[6] Joeran Beel, Min-Yen Kan, and Moritz Baumgart. 2025. Evaluating Sakana's AI Scientist: Bold Claims, Mixed Results, and a Promising Future? *ACM SIGIR Forum* 59, 1 (2025), 1–20. doi:10.1145/3769733.3769747

[7] Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, et al. 2026. Accelerating scientific discovery with Co-Scientist. *Nature* (2026). doi:10.1038/s41586-026-10644-y

[8] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. 2024. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. *arXiv preprint arXiv:2408.06292* (2024). arXiv:2408.06292 [cs.AI]

[9] Nasim Sonboli, Masoud Mansoury, Ziyue Guo, Shreyas Kadekodi, Weiwen Liu, Zijun Liu, Andrew Schwartz, and Robin Burke. 2021. LibRec-Auto: A Tool for Recommender Systems Experimentation. In *Proceedings of the 30th ACM International Conference on Information and Knowledge Management*. 4584–4593. doi:10.1145/3459637.3482006

[10] Tobias Vente, Michael D. Ekstrand, and Joeran Beel. 2023. Introducing LensKit-Auto, an Experimental Automated Recommender System (AutoRecSys) Toolkit. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 1212–1216. doi:10.1145/3604915.3610656

[11] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. 2025. The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search. *arXiv preprint arXiv:2504.08066* (2025). arXiv:2504.08066 [cs.AI]

## A EXPERIMENTAL PROMPTS

### A.1 Phase 1: Complex Multi-Dataset Prompt

"I'd like to run an experiment to quantify how much data split random seeds affect recommender system accuracy. Please use LensKit 0.14.4 to test three algorithms: ALS, ItemKNN, and Pop. Run this on the following three datasets with implicit feedback: MovieLens100K, Amazon Video Games, Last.FM. The raw files are stored in your working directory with the filenames u.data, VideoGames.csv, user_taggedartists-timestamps.dat. First, preprocess all datasets with 5-core filtering. For the Amazon and MovieLens datasets, please also convert any ratings greater than 3 to implicit interactions. Here's the main experimental procedure: Generate 5 different random seeds for data splitting. For each algorithm, dataset, and seed, please do a user-based 80/20 holdout split. Train all models using standard hyperparameters. For the analysis, I need you to measure nDCG@k and Precision@k for k=1, 5, 10 and conduct a short statistical analysis."

### A.2 Phase 1: Simplified Exploratory Data Analysis Prompt

"Analyze the provided MovieLens100k dataset u.data (Format: user_id | item_id | rating | timestamp) to identify user behavior patterns. Confirm the total number of unique user_ids and unique item_ids identified in the file. Identify the Top 3 most active users (by count of Ratings). For the #1 most active user, calculate their mean rating score. Identify Item 50 and Item 181. Calculate the "co-occurence": How many users rated both items? Are there any timestamps that fall outside the expected range for this 1997-1998 dataset? Do not guess. Use the full context provided."

### A.3 Phase 2: Standardized Pipeline Prompt

"I'd like to run an experiment to quantify how much data split random seeds affect recommender system accuracy. Please test two algorithms: ALS and Pop. Run this on the MovieLens100K dataset with implicit feedback. First, preprocess the dataset with 5-core filtering and convert any ratings greater than 3 to implicit interactions. Generate 5 different random seeds for data splitting. For each algorithm and seed, do a user-based 80/20 holdout split and train the models using standard hyperparameters. Measure nDCG@10 and Precision@10 and conduct a short statistical analysis across seeds."

## B COMPARISON WITH THE ORIGINAL AUTORECLAB CASE STUDY

**Table 3: Comparison of the original AutoRecLab case study and our Phase-1 evaluation.**

| Feature | Original case study | Our Phase 1 |
|---|---|---|
| Models | GPT-5 | GPT-5-nano, GPT-5.4-mini, and GPT-5.4 |
| Prompts | Four runs with slightly different prompts and variables | One complex prompt and one simplified exploratory prompt |
| Search configuration | 10 or 20 tree-search iterations | 3 draft nodes, 10 iterations, temperature 1, debug probability 0.3, and epsilon 0.3 |
| Datasets | MovieLens100K, Amazon Video Games, and Last.FM | The same three datasets for the complex prompt; MovieLens100K for the simplified prompt |
| End-to-end status | 3 of 4 runs technically completed | GPT-5.4: 1/6; GPT-5.4-mini: 0/2; GPT-5-nano: not assessable from the preserved archive |
| Environment | LensKit 0.14.4 and the original experimental software environment | The pinned Phase-1 environment documented above |

## C RUN-LEVEL EXECUTION AND VALIDATION OVERVIEW

**Table 4: Run-level execution and result status. Each row represents one AutoRecLab invocation.**

| Run | Phase / Task | Configuration | Tag / Commit | Search | Started | Executable Node | Pipeline | Task | Artifact coverage | Evaluation | Resources |
|---|---|---|---|---|---|---|---|---|---|---|---|
| run_001 | P1 / Simple | GPT-5-nano | v0.0.1 | 3 nodes / 10 iter. | Yes | Yes | Yes | Complete | 6/6 | Inspected; no independent reference | 14 min / $0.13 |
| run_002 | P1 / Simple | GPT-5-nano | v0.0.1 | 3 nodes / 10 iter. | Yes | Yes | Yes | Complete | 6/6 | Dataset-path fallback; use with caution | 14 min / $0.04 |
| run_003 | P1 / Simple | GPT-5.4-mini | v0.0.1 | 3 nodes / 10 iter. | Yes | Yes | Yes | Complete | 6/6 | Inspected; no independent reference | 1.5 min / $0.06 |
| run_006 | P1 / Complex | GPT-5.4-mini | v0.0.1 | 3 nodes / 10 iter. | Yes | No | No | None | 0/45 | Not usable; LensKit API failure | 5.5 min / $0.25 |
| run_007 | P1 / Complex | GPT-5.4-mini | v0.0.1 | 3 nodes / 10 iter. | Yes | No | No | None | 0/45 | Not usable; syntax/API failures | 11 min / $0.20 |
| run_008 | P1 / Complex | GPT-5.4 | v0.0.1 | 3 nodes / 10 iter. | Yes | Yes | No | Partial | 45/45[a] | Not usable; final run failed | 20.5 min / $1.00 |
| run_009 | P1 / Complex | GPT-5.4 | v0.0.1 | 3 nodes / 10 iter. | Yes | Yes | No | Partial | 0/45 | Not usable; timeout/interpreter crash | >60 min / $0.42 |
| run_010 | P1 / Complex | GPT-5.4 | v0.0.1 | 3 nodes / 10 iter. | Yes | No | No | Partial | 45/45[a] | Not usable; final run failed | 39 min / $1.07 |
| run_011 | P1 / Complex | GPT-5.4 | v0.0.1 | 3 nodes / 10 iter. | Yes | No | No | None | 0/45 | Not usable; LensKit API failure | 20 min / $0.70 |
| run_012 | P1 / Complex | GPT-5.4 | v0.0.1 | 3 nodes / 10 iter. | Yes | Yes | No | Partial | 0/45 | Not usable; final run failed | 18 min / $0.50 |
| run_013 | P1 / Complex | GPT-5.4 | v0.0.1 | 3 nodes / 10 iter. | Yes | Yes | Yes | Complete | 45/45 | Inspected; no independent reference | 60 min / $1.50 |
| run_014 | P1 / Simple | GPT-5.4-mini | v0.0.1 | 3 nodes / 10 iter. | Yes | Yes | Yes | Complete | 6/6 | Inspected; no independent reference | 1.5 min / $0.05 |
| run_015 | P1 / Simple | GPT-5.4-mini | v0.0.1 | 3 nodes / 10 iter. | Yes | Yes | Yes | Complete | 6/6 | Inspected; no independent reference | 1 min / $0.05 |
| run_016 | P1 / Simple | GPT-5.4 | v0.0.1 | 3 nodes / 10 iter. | No | No | No | None | 0/6 | Not usable; process failed to start | <1 min / $0.00 |
| run_017 | P1 / Simple | GPT-5.4 | v0.0.1 | 3 nodes / 10 iter. | Yes | Yes | Yes | Complete | 6/6 | Inspected; no independent reference | 2.5 min / - |
| R1 | P2 / Standardized | Mixed | 43673ac | 2 draft / 8 iter. / 2 refine. / 13 nodes | Yes | Yes | Yes | Complete | 10/10[b] | Artifact-complete; inspected; | 43.95 min / $2.38 |
| R2 | P2 / Standardized | Mixed replica | 43673ac | 2 draft / 8 iter. / 2 refine. / 13 nodes | Yes | Yes | Yes | Complete | 10/10[b] | Artifact-complete; inspected; | 43.93 min / $2.39 |
| R3 | P2 / Standardized | Solo terra (all roles) | 43673ac | 2 draft / 8 iter. / 2 refine. / 13 nodes | Yes | Yes | Yes | Complete | 10/10[b] | Artifact-complete; inspected; | 62.33 min / $9.84 |
| R4 | P2 / Standardized | Solo luna (all roles) | 43673ac | 2 draft / 8 iter. / 2 refine. / 13 nodes | Yes | Yes | Yes | Complete | 10/10[b] | Artifact-complete; inspected; | 46.18 min / $3.64 |

[a] The artifact contained 45 result rows, but the corresponding AutoRecLab invocation did not terminate as an accepted complete run. The complex task required 45 dataset–algorithm–seed combinations (3 datasets ×3 algorithms ×5 seeds). For the simple prompt, six requested checks were expected.

[b] For Phase 2, artifact coverage counts algorithm–split-seed rows: 2 algorithms × 5 seeds = 10. Each row contains both nDCG@10 and Precision@10. "Complete" refers to artifact completeness, not independent validation of preprocessing, splitting, or metric computation.